

C Programming Notes — SCS 1101 (NUST)

Introduction to Computer Science and Programming: Explained Like You're 5

National University of Science and Technology · Faculty of Engineering · SCS 1101

2 October 2026

Contents

How to use these notes · Part 1 Programs and compilers · Part 2 Your first C program · Part 3 Variables and data types · Part 4 Operators · Part 5 Input and output · Part 6 Decisions · Part 7 Loops · Part 8 Functions and recursion · Part 9 Arrays and strings · Part 10 Pointers and memory · Part 11 Structs and data structures · Part 12 Algorithms · Part 13 Files · Part 14 Bugs and undefined behaviour · Practice paper with full solutions · Cheat sheet and study plan · References

How to use these notes

These notes cover every C-related topic in the SCS 1101 outline (Topics 7 to 10, plus the data-structure and algorithm parts) and teach each one twice: first like you are 5, then the proper exam version.

The course in one table (from the SCS 1101 outline, lecturer Mr B. Guvava, office FD60):

Item	What it means for you
Format	5-week intensive, 08:00 to 16:00, computer lab
Language	C (the outline names C for all practical work)
Continuous assessment	40%: Test 1 (10%), Assignment 1 (10%), Assignment 2 (10%), Group research (10%)
Final exam	60%, 3 hours
Rules	No exam entry without coursework; at least 80% attendance; no plagiarism
Consultation	Weekdays 12:00 to 15:30, Tue/Thu preferred, book by email

How every part is built:

1. **Like you're 5** — a picture-in-words with a toy, a kitchen or a school example.
2. **The proper version** — the definitions and rules an examiner wants to see.
3. **Code** — a short program you can type and run.
4. **Diagram or flowchart** — where the idea has a shape.
5. **Check yourself** — quick questions. Full exam-style questions with worked solutions are near the end.

A note on the past papers you uploaded: SCS 1100 (IT and Computer Applications, Dec 2024) and SCS 1103 (Operating Systems, Nov 2024) are not C papers. Their style is still useful: "define", "differentiate using examples", "discuss five" and a calculation question worth 10. The practice section copies that style for C, and reuses the scheduling and Excel-IF ideas from them because SCS 1101 also covers operating systems and decision logic.

Sources: MIT OpenCourseWare C courses, peer-reviewed and arXiv research on C errors and memory safety, the NUST resources named in your outline, and the outline's own textbooks (Kernighan & Ritchie; Hanly & Koffman). All are listed with links in References.

Part 1 — What a program is, and how C becomes something the computer runs

A program is a list of exact instructions. C is a high-level language that a **compiler** translates into machine code *before* the program runs.

Like you're 5. The computer is a super-fast robot that only understands light switches: on (1) and off (0). You speak English. A program is a recipe for the robot. Writing it in 0s and 1s would take forever, so you write it in C, which looks like a mix of English and maths. A **compiler** is a translator who takes your whole recipe book, translates all of it into robot language, and hands it over in one go. An **interpreter** is a translator who stands beside the robot and translates one line at a time while it cooks.

The proper version — three levels of language (Topic 7.1–7.2):

Level	Example	Translated by	Key facts
Machine code (low)	`10110000 01100001`	Nothing — the CPU reads it directly	Pure binary; different for each CPU family; fastest; very hard for humans
Assembly (low)	`MOV AL, 61h`	Assembler	Short words (mnemonics), roughly one per machine instruction; still CPU-specific
High-level	C, Fortran, Python	Compiler or interpreter	Readable and portable; one line becomes many machine instructions

C is often called a "middle-level" language: it reads like a high-level language but lets you work with memory addresses directly (pointers, Part 10). MIT's 6.087 Practical Programming in C describes C as the main language behind UNIX and the usual language for embedded processors and micro-controllers.

Compiler vs interpreter (Topic 7.3) — a favourite 5-mark question:

	Compiler	Interpreter
How it translates	Whole program at once, before running	One statement at a time, while running
Output	A separate executable file	No separate file; needs the interpreter every time
Running speed	Fast	Slower
Errors	Listed together after compiling; nothing runs until fixed	Program stops at the first bad line
Examples	C, C++, Fortran	Python, JavaScript, BASIC

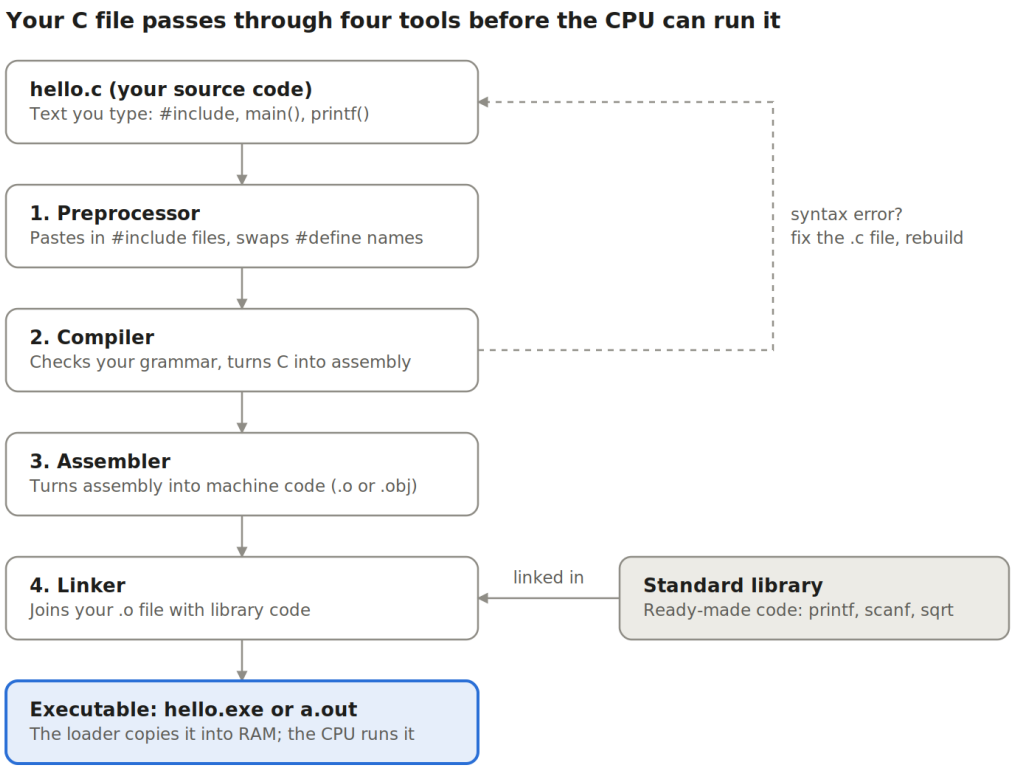


Figure 1 — The C build pipeline: source code to executable in four tools

Read it top to bottom: most of your errors appear at the compiler (grammar mistakes), and errors like "undefined reference to printf" come from the linker.

Why a civil engineer cares: your later NUST modules include Computational Techniques and the Finite Element Method, and the solvers behind those are compiled programs built exactly this way.

Check yourself

1. Why can't the CPU run `hello.c` directly? — *It only understands machine code, so the file must be compiled, assembled and linked first.*
2. Which tool joins your code to the code for `printf`? — *The linker.*
3. Give two differences between a compiler and an interpreter. — *Any two rows of the table above.*

Part 2 — Your first C program (Topics 7.4, 10.1, 10.4)

Every C program has the same skeleton: borrow tools with `#include`, start in `main()`, end each statement with `;`, and finish with `return 0;`.

Like you're 5. A C program is like a letter. At the top you say which toolbox you are borrowing (`#include <stdio.h>`). `main` is the front door: the computer always walks in there first. Curly braces `{ }` are a lunchbox that holds the steps. Every sentence ends with a semicolon instead of a full stop. `return 0;` is you shouting "All done, everything is fine!"

```c
/* hello.c - my first C program
   Purpose: print a greeting */
#include <stdio.h>          // borrow the standard input/output toolbox

int main(void)              // the computer always starts here
{
    printf("Hello, NUST!\n");  // print text; \n means new line
    return 0;               // tell the operating system: finished OK
}
```

Line	What it does
`/* ... */` and `// ...`	Comments. The compiler ignores them; they are notes for humans
`#include <stdio.h>`	Preprocessor directive: pastes in the declarations for `printf` and `scanf`
`int main(void)`	The function where execution begins; `int` means it returns a whole number
`{ ... }`	A block: groups statements together
`printf(...);`	A statement that calls a library function to print
`return 0;`	Ends `main` and reports success (0) to the operating system

Build and run (GCC, the usual lab compiler on Linux and inside Code::Blocks/Dev-C++ on Windows):

```
gcc hello.c -o hello      # compile + link into a program called hello
./hello                   # run it (on Windows: hello.exe)
```

Grammar, syntax and keywords (Topic 7.4).

- **Syntax** = the spelling and grammar rules (where semicolons and brackets go). Break them and the compiler refuses.
- **Semantics** = the meaning. A program can have perfect syntax and still do the wrong thing (a logic error).
- **Keywords** = words C reserves for itself. You may not use them as names. C89 has 32 of them:

```
auto     break    case     char     const    continue default  do
double   else     enum     extern   float    for      goto     if
int      long     register return   short    signed   sizeof   static
struct   switch   typedef  union    unsigned void     volatile while
```

- **Identifiers** = names you invent (`total`, `flow_rate`). Rules: letters, digits and `_` only; must not start with a digit; no spaces; not a keyword; case-sensitive, so `Area` and `area` are different.

Name	Valid?	Why
`pipe_length`	Yes	Letters and underscore
`_count2`	Yes	May start with `_`
`2ndBeam`	No	Starts with a digit
`flow rate`	No	Contains a space
`float`	No	It is a keyword

Escape sequences you will use in `printf`: `\n` new line, `\t` tab, `\\` a backslash, `\"` a double quote, `%%` a percent sign.

Coding standards (Topic 10.4): indent the inside of every `{ }` by 4 spaces, one statement per line, meaningful names (`area` not `a`), and a comment at the top saying what the program does. Markers reward this: MIT's 6.087 syllabus awards marks for coding style and clarity, not just correctness.

What beginners actually get wrong. A Kyushu University study of 53,505 compiler errors from 909 novice C students found that most were simple, repeated types, such as using a variable that was never declared (Fu et al., 2015). So when you see an error, check these first: a missing `;`, a misspelt name (`Printf` instead of `printf`), a variable used before it is declared, and a missing `#include`.

Check yourself

1. What is wrong with `int 2total;`? — *An identifier cannot start with a digit.*
2. Why does `printf` need `#include <stdio.h>`? — *The header tells the compiler what `printf` looks like before you use it.*
3. Differentiate a syntax error from a logic error. — *A syntax error breaks the language rules and stops compilation; a logic error compiles and runs but gives a wrong answer.*

Part 3 — Variables, data types and how data sits in memory (Topics 2, 10.2)

A variable is a named piece of memory; its **type** decides what kind of value it holds and how many bytes it takes.

Like you're 5. Picture jars on a kitchen shelf. The sticker on the jar is the **name**. What is inside is the **value**. The **type** says what may go in: a sugar jar (whole numbers), a flour jar (decimals) or a sweet jar (one letter). A big jar takes more shelf space, just like a `double` takes more memory than a `char`.

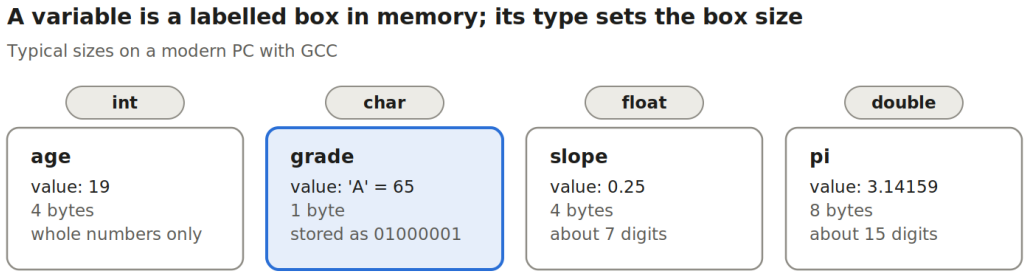


Figure 2 — Four variables as memory boxes: type, value and size

The `char` box is highlighted because it links straight to Topic 2: a letter is really a number (its ASCII code) stored in binary.

Three words examiners love (Topic 10.2):

```c
int age;          /* declaration: create the box (contents = garbage!) */
int count = 0;    /* initialization: create AND fill it at once */
age = 19;         /* assignment: put a new value in an existing box */
```

An uninitialised local variable holds whatever leftover bits were in that memory, so always initialise counters and totals.

The basic data types:

Type	Holds	Typical size	`printf`/`scanf` code	Example
`char`	One character (really a small integer)	1 byte	`%c`	`char grade = 'A';`
`int`	Whole numbers, about ±2.1 billion	4 bytes	`%d`	`int bags = 50;`
`float`	Decimals, about 7 significant digits	4 bytes	`%f`	`float slope = 0.25f;`
`double`	Decimals, about 15 significant digits	8 bytes	`%f` print, `%lf` scan	`double g = 9.81;`

Type	Holds	Typical size	`printf`/`scanf` code	Example
`long`, `short`, `unsigned`	Modifiers that change an int's range	varies	`%ld`, `%hd`, `%u`	`unsigned int n = 40000;`

Check sizes on your own machine with `sizeof`: `printf("%zu\n", sizeof(double));` prints `8` on a typical PC.

How the bits are stored (ties to Topics 2.1–2.3):

- **Integers** are stored in binary. 19 ÷ 2 repeatedly gives remainders 1, 1, 0, 0, 1, read upwards: 19 = 10011_2 = 13_{16}.
- **Negative integers** use two's complement. For −5 in 8 bits: 5 = 00000101 → flip the bits 11111010 → add 1 → **11111011**.
- **Characters** are stored as codes. `'A'` = 65 = 0x41 = 01000001; `'a'` = 97; `'0'` = 48. So `'A' + 1` is `'B'`.
- **Floats** use the IEEE 754 format (sign, exponent, mantissa). Many decimals, like 0.1, cannot be stored exactly, so never test two floats with `==`; check whether their difference is tiny instead.

Constants — values that must not change:

```c
#define PI 3.14159        /* preprocessor swaps the word PI for the number */
const double G = 9.81;    /* a typed variable the compiler won't let you change */
```

Check yourself

1. What does `int x;` contain before you assign to it? — *Unknown garbage; never rely on it.*
2. Convert 45 to binary and hex. — *101101_2 and $2D_{16}$.*
3. Why is `char c = 'B'; printf("%d", c);` legal? — *A `char` is a small integer, so it prints its ASCII code, 66.*

Part 4 — Operators and expressions

Operators are the maths and comparison symbols of C; the two traps that cost the most marks are integer division and mixing up `=` with `==`.

Like you're 5. You have 7 sweets and 2 friends. Each friend gets 3 whole sweets (`7 / 2` is `3`), and 1 sweet is left over (`7 % 2` is `1`). C does not cut sweets in half when both numbers are whole numbers. If you want halves, at least one number must be a decimal: `7.0 / 2` is `3.5`.

Arithmetic operators:

Operator	Meaning	`int` example	Result
`+ - *`	Add, subtract, multiply	`4 * 3`	`12`
`/`	Divide (whole-number division if both sides are `int`)	`7 / 2`	`3`
`%`	Remainder (modulus); `int` only	`7 % 2`	`1`
`(double)`	Cast: treat a value as another type	`(double)7 / 2`	`3.5`

Assignment shortcuts: `x += 5;` means `x = x + 5;`. The same works for `-=`, `*=`, `/=`, `%=`.

Increment and decrement: `x++` uses x, *then* adds 1 (post-increment). `++x` adds 1 *first*, then uses it (pre-increment).

```c
int x = 5;
int y = x++ + 2;   /* uses 5, so y = 7; afterwards x = 6 */
int z = ++x * 2;   /* x becomes 7 first, so z = 14 */
```

Relational and logical operators (results are `1` for true, `0` for false):

Operator	Meaning	Example (a = 5, b = 8)	Result
`== !=`	Equal, not equal	`a == b`	`0`
`< > <= >=`	Comparisons	`a < b`	`1`
`&&`	AND: both must be true	`a > 0 && b > 0`	`1`
`||`	OR: at least one true	`a > 6 || b > 6`	`1`
`!`	NOT: flips true/false	`!(a == 5)`	`0`

Short-circuit rule: in `a != 0 && b / a > 2`, if `a` is 0 the right side is never evaluated, so there is no division by zero. Use this on purpose.

The = vs == trap: `if (x = 5)` *assigns* 5 to x and is always true. You meant `if (x == 5)`. Compile with `gcc -Wall` and the compiler warns you.

Precedence (highest first) — learn the top five:

1. `()` brackets
2. `! ++ --` unary minus, casts
3. `* / %`
4. `+ -`
5. `< <= > >=`, then `== !=`
6. `&&`, then `||`
7. `= += -=` (assignment, done last, right to left)

Worked example — evaluate `2 + 3 * 4 % 5 - 6 / 4` (all `int`):

1. `* / %` first, left to right: `3 * 4 = 12`, then `12 % 5 = 2`, then `6 / 4 = 1`.
2. Now `2 + 2 - 1`, left to right = **3**.

Engineering example — flow in a pipe, Q = A × v with A = πd²/4:

```c
#include <stdio.h>
#define PI 3.14159

int main(void)
{
    double d = 0.3;              /* pipe diameter in m */
    double v = 1.5;              /* velocity in m/s */
    double area = PI * d * d / 4.0;
    double Q = area * v;         /* flow in m^3/s */
    printf("Q = %.4f m^3/s\n", Q);  /* prints Q = 0.1060 m^3/s */
    return 0;
}
```

Check yourself

1. `int a = 17, b = 5;` What are `a / b`, `a % b` and `(double)a / b`? — *3, 2 and 3.4.*
2. Why is `if (n = 0)` a bug? — *It assigns 0, which counts as false, so the block never runs and n is wiped.*
3. Evaluate `10 - 4 / 3 * 2`. — *`4 / 3 = 1`, `1 * 2 = 2`, `10 - 2 = 8`.*

Part 5 — Input and output (Topic 10.3)

`printf` writes to the screen and `scanf` reads from the keyboard; both use format codes such as `%d`, and `scanf` needs an `&` in front of each ordinary variable.

Like you're 5. `printf` is the computer *talking*. `scanf` is the computer *listening*. When it listens, it needs to know which jar to drop your answer into, so you hand it the jar's address with `&`. Forget the `&` and it throws your answer at a random spot on the shelf.

Format codes:

Code	Prints / reads	Example	Output
`%d`	`int` in decimal	`printf("%d", 42);`	`42`
`%f`	`float`/`double` (6 decimals)	`printf("%f", 2.5);`	`2.500000`
`%.2f`	2 decimal places	`printf("%.2f", 3.14159);`	`3.14`
`%lf`	`double` in `scanf`	`scanf("%lf", &L);`	—
`%e`	Scientific notation	`printf("%e", 21000.0);`	`2.100000e+04`
`%c`	One character	`printf("%c", 'Y');`	`Y`
`%s`	A string	`printf("%s", "NUST");`	`NUST`
`%o` / `%x`	Octal / hex (Topic 2)	`printf("%o %x", 45, 45);`	`55 2d`
`%5d` / `%-8s`	Width 5 right-aligned / width 8 left-aligned	`printf("[%5d]", 42);`	`[   42]`

Worked program — maximum bending moment of a simply supported beam with a uniform load, $M = wL^2/8$:

```c
#include <stdio.h>

int main(void)
{
    double w, L, M;                      /* load kN/m, span m, moment kNm */

    printf("Enter load w (kN/m) and span L (m): ");
    if (scanf("%lf %lf", &w, &L) != 2) { /* scanf returns how many it read */
        printf("Invalid input\n");
        return 1;
    }

    M = w * L * L / 8.0;
    printf("Max bending moment = %.2f kNm\n", M);
```

```
    return 0;
}
```

Sample run: entering `12 6` prints `Max bending moment = 54.00 kNm`.

Four I/O rules that save marks:

1. `scanf` needs `&` before `int`, `float`, `double` and `char` variables, but **not** before an array name (`scanf("%s", name);`), because an array name already is an address (Part 10).
2. Read a `double` with `%lf`, but print it with `%f`.
3. To read a single character after a number, put a space first: `scanf(" %c", &ans);`. The space swallows the leftover Enter key.
4. `scanf` returns the number of items it read. Checking it (as above) is what good programmers do.

Check yourself

1. What does `printf("%6.2f|", 3.14159);` print? — *`3.14|` (width 6, 2 decimals, so two leading spaces).*
2. Spot the bug: `int n; scanf("%d", n);` — *Missing `&`; it should be `&n`.*
3. Print 255 in decimal, octal and hex. — *`printf("%d %o %x", 255, 255, 255);` gives `255 377 ff`.*

Part 6 — Making decisions: if, else if, switch (Topic 10.5)

Decisions let a program pick a path: `if` for one test, an `if … else if … else` ladder for ranges, and `switch` for choosing between exact values.

Like you're 5. You reach a road junction. A sign asks: "Is it raining?" If yes, take the left road to the bus stop. If no, take the right road and walk. An `if` is that sign. A ladder of `else if` signs is a row of junctions: you stop at the first one that says yes and ignore the rest.

The three shapes:

```c
if (marks >= 50)                  /* 1. one test */
    printf("Pass\n");

if (marks >= 50)                  /* 2. two paths */
    printf("Pass\n");
else
    printf("Fail\n");

if (t > 30)                       /* 3. a ladder: first true test wins */
    printf("Very hot\n");
else if (t >= 25)
    printf("Hot\n");
else if (t > 20)
    printf("Mild\n");
else if (t >= 15)
    printf("Slightly cold\n");
else
    printf("Cold\n");
```

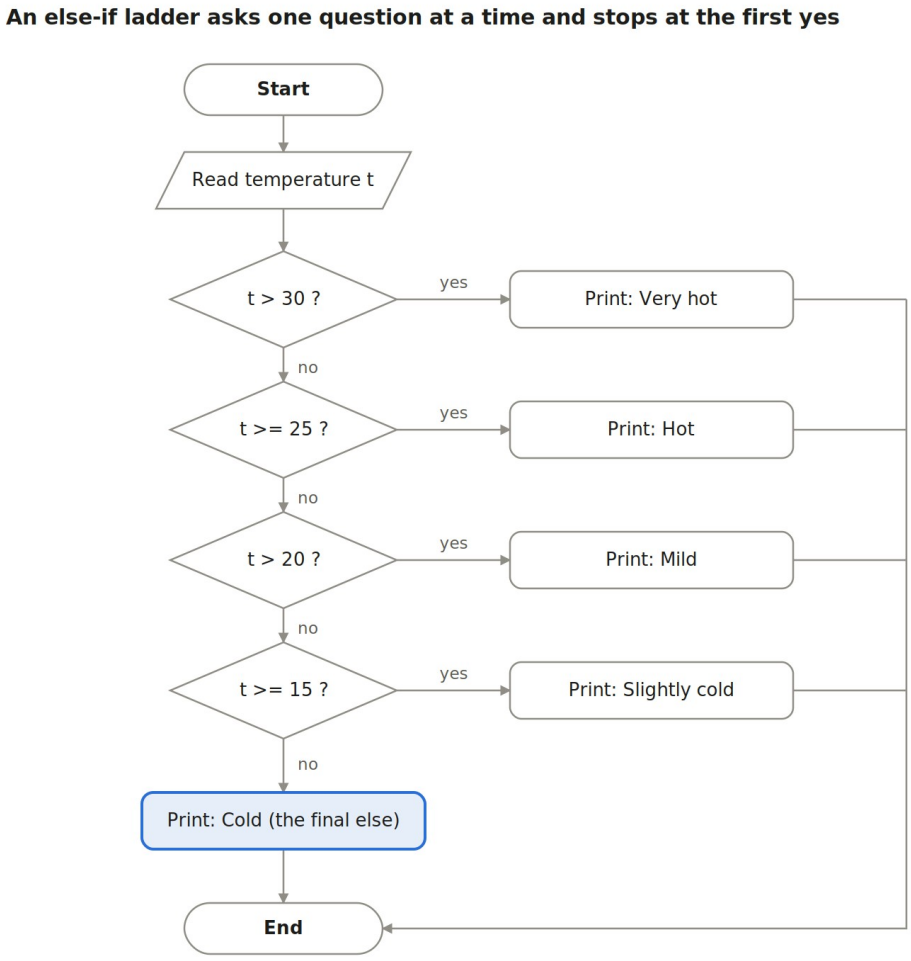


Figure 3 — Else-if ladder flowchart for the temperature classification

Each “no” drops to the next question, so later tests can assume the earlier ones failed: by the time you ask `t > 20`, you already know `t < 25`.

Link to your SCS 1100 paper (Question 5). That question asks for the same ladder as an Excel formula. The figure puts 20 in “Slightly cold”, so the boundaries that match the figure are the ones used above. In Excel, with the first temperature in A2:

```
=IF(A2>30,"Very hot",IF(A2>=25,"Hot",IF(A2>20,"Mild",IF(A2>=15,"Slightly
cold","Cold"))))
```

Exam tip: when ranges overlap at the edges (“20 to 25 inclusive” and “15 to 20 inclusive”), state which side you gave the boundary to and check it against any sample data.

`switch` — for exact values (int or char), not ranges:

```c
char grade;
printf("Enter grade (A-D): ");
scanf(" %c", &grade);
```

```c
switch (grade) {
    case 'A': printf("Distinction\n");  break;
    case 'B': printf("Merit\n");        break;
    case 'C': printf("Pass\n");         break;
    case 'D': printf("Fail\n");         break;
    default:  printf("Invalid grade\n");
}
```

Forget a `break` and execution **falls through** into the next case. `default` catches anything not listed.

Also know:

- **Braces:** without `{ }`, only the *one* statement after `if` belongs to it. Always use braces when there is more than one statement.
- **Nested if:** an `if` inside another `if`, for example check `if (age >= 18)` and then, inside it, `if (hasLicence)`.
- **Conditional operator:** `max = (a > b) ? a : b;` is a one-line if-else.

Check yourself

1. When would you choose `switch` over an `if` ladder? — *When comparing one variable against exact int or char values, such as a menu choice.*
2. What prints for `t = 25` in the ladder above? — *Hot.*
3. Write a one-line expression for the absolute value of x. — *`(x < 0) ? -x : x`.*

Part 7 — Loops: for, while, do-while (Topic 10.5)

A loop repeats a block: use `for` when you know how many times, `while` when you repeat until something changes, and `do-while` when the block must run at least once.

Like you're 5.

- `for` = "Clap your hands 5 times." You know the count before you start.
- `while` = "Keep stirring the porridge *while* it is lumpy." You check first; if it is already smooth, you never stir.
- `do-while` = "Taste the soup, *then* decide if it needs salt." You always taste at least once.

while tests before the body; do-while runs the body once before testing

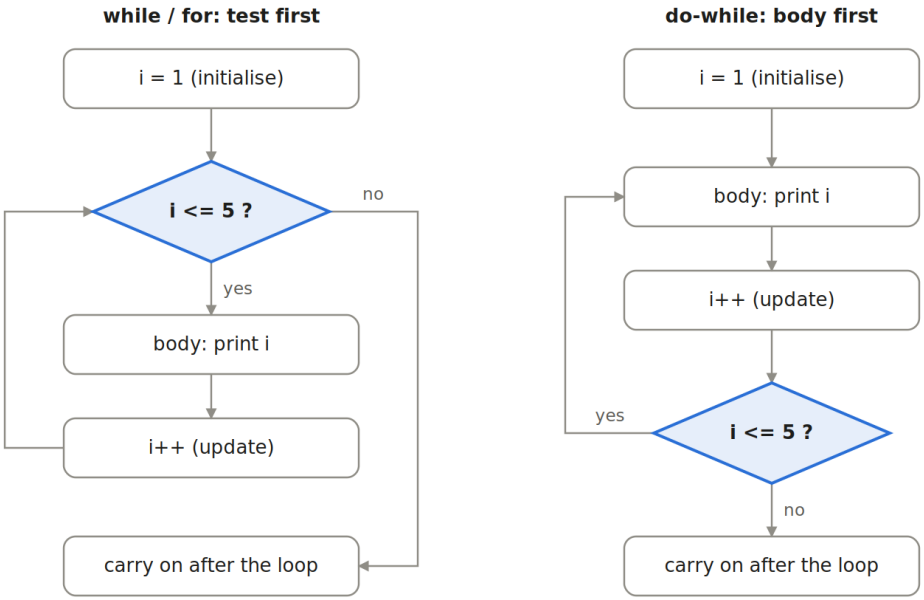


Figure 4 — while vs do-while: where the test sits

The highlighted diamond is the test. On the left it comes first, so the body may run zero times. On the right it comes last, so the body always runs at least once.

`for` — counting loop. The three parts in the brackets are the initialise, test and update boxes from the left-hand flowchart:

```
int i, sum = 0;
for (i = 1; i <= 5; i++) {   /* initialise; test; update */
    sum += i;
}
printf("Sum = %d\n", sum);   /* Sum = 15 */
```

Trace table — examiners love these. Fill one row per pass:

Pass	i at test	i <= 5?	sum after body
1	1	yes	1
2	2	yes	3
3	3	yes	6
4	4	yes	10
5	5	yes	15
—	6	no, exit	15

`while` — sentinel loop. Read daily rainfall readings until the user types −1, then print the average:

```c
double r, total = 0.0;
int days = 0;

printf("Rainfall (mm), -1 to stop: ");
scanf("%lf", &r);
while (r != -1) {
    total += r;
    days++;
    scanf("%lf", &r);           /* read the next value, or the loop never ends */
}
if (days > 0)
    printf("Average = %.1f mm over %d days\n", total / days, days);
```

`do-while` — input validation. Keep asking until the menu choice is 1 to 3:

```c
int choice;
do {
    printf("1. Area  2. Volume  3. Quit\nChoice: ");
    scanf("%d", &choice);
} while (choice < 1 || choice > 3);   /* note the semicolon */
```

`break` and `continue`: `break` leaves the loop at once. `continue` skips the rest of this pass and jumps to the next test.

```c
for (i = 1; i <= 10; i++) {
    if (i % 2 == 0) continue;   /* skip even numbers */
    if (i > 7) break;           /* stop completely after 7 */
    printf("%d ", i);           /* prints 1 3 5 7 */
}
```

Nested loops — a loop inside a loop. The inner one finishes completely for every single pass of the outer one:

```c
for (int row = 1; row <= 3; row++) {
    for (int col = 1; col <= row; col++)
        printf("*");
    printf("\n");
}
/* prints:
*
```

```
**
***          */
```

Three loop bugs to avoid:

1. **Infinite loop** — the update is missing, so the test never becomes false (`while (i <= 5)` with no `i++`).
2. **Off-by-one** — `i < 5` runs 4 times starting from 1; `i <= 5` runs 5 times. Trace the first and last pass.
3. **Stray semicolon** — `for (i = 0; i < 5; i++);` loops an *empty* statement, and the block below runs once.

Check yourself

1. How many times does `for (i = 10; i > 0; i -= 3)` run, and with which values? — *4 times: 10, 7, 4, 1.*
2. Rewrite `for (i = 0; i < n; i++) printf("%d", i);` as a while loop. — *`i = 0; while (i < n) { printf("%d", i); i++; }`*
3. Give one situation that needs `do-while`. — *Showing a menu or asking for input that must happen at least once before it can be checked.*

Part 8 — Functions and recursion (Topic 8.4)

A function is a named, reusable block that takes inputs (parameters) and can hand back one result (`return`). Recursion is a function that solves a problem by calling itself on a smaller version, stopping at a base case.

Like you're 5. A function is a vending machine. You put in coins (the **arguments**), press the button (the **call**), and a drink comes out (the **return value**). You don't need to know what happens inside. Recursion is a set of Russian dolls: open a doll and there is a smaller one inside, and another, until the tiniest doll that doesn't open. That tiny doll is the **base case**.

The three pieces of a function:

```c
#include <stdio.h>

double moment(double w, double L);        /* 1. prototype: promise it exists */

int main(void)
{
    double M = moment(12.0, 6.0);         /* 2. call: arguments 12 and 6 */
    printf("M = %.2f kNm\n", M);          /* M = 54.00 kNm */
    return 0;
}

double moment(double w, double L)         /* 3. definition */
{                                         /* w, L are parameters */
    return w * L * L / 8.0;               /* send the answer back */
}
```

Term	Meaning
Return type	Type of the answer (`double` here); `void` means no answer
Parameters	Variables in the definition that receive the inputs
Arguments	The actual values passed in the call
Prototype	The function's header ending in `;`, placed before `main` so the compiler knows it

Pass by value. C gives a function *copies* of the arguments. Changing a parameter does not change the caller's variable. That is why this `swap` fails, and Part 10 fixes it with pointers:

```c
void swap(int a, int b) { int t = a; a = b; b = t; }  /* swaps the copies only */
```

Scope — who can see a variable:

- **Local** variables live inside a function's `{ }` and vanish when it returns.
- **Global** variables are declared outside all functions; every function can see them. Use them sparingly.
- A **`static`** local keeps its value between calls, for example a call counter.

Recursion. Every recursive function needs two parts: a **base case** that answers directly, and a **recursive case** that calls itself on a smaller problem.

```c
long fact(int n)
{
    if (n <= 1)            /* base case */
        return 1;
    return n * fact(n - 1);  /* recursive case */
}
```

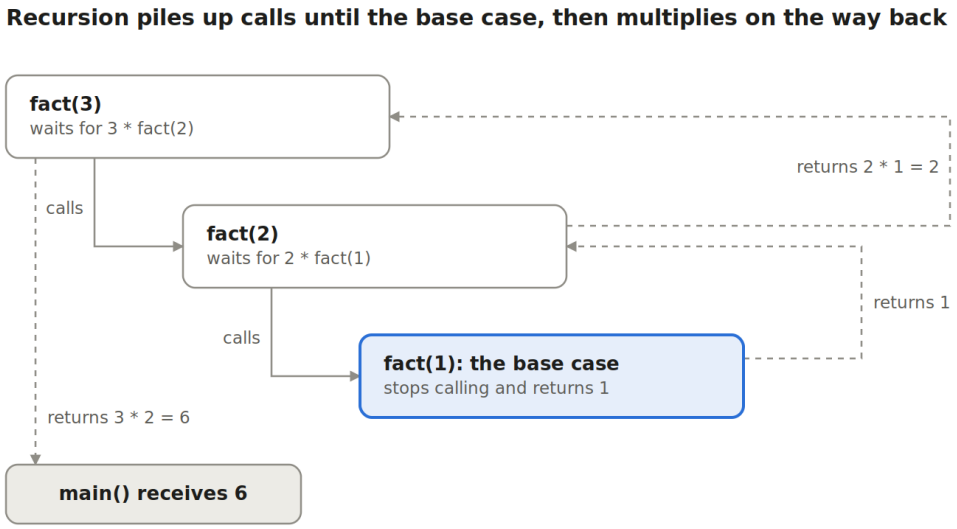


Figure 5 — The call stack for fact(3): three calls down, three returns up

Each call waits on the **call stack** until the one below it answers. The dashed arrows are the answers travelling back up: 1, then 2 × 1 = 2, then 3 × 2 = 6. Without a base case the calls never stop, the stack fills, and the program crashes with a **stack overflow**.

More recursive classics:

```c
int fib(int n)        { return (n < 2) ? n : fib(n - 1) + fib(n - 2); }  /* 0 1 1 2 3 5 8 */
int sumDigits(int n)  { return (n == 0) ? 0 : n % 10 + sumDigits(n / 10); }  /* 472 -> 13 */
int power(int b, int e) { return (e == 0) ? 1 : b * power(b, e - 1); }   /* 2^5 = 32 */
```

Recursion vs iteration (loops):

	Recursion	Iteration
How it repeats	Function calls itself	`for` / `while` loop
Stops when	Base case is reached	Loop test becomes false
Memory	One stack frame per call, more memory	One set of variables

	Recursion	Iteration
Best for	Naturally self-similar problems: trees, divide and conquer	Simple counting and repetition

Check yourself

1. Name the two parts every recursive function must have. — *A base case and a recursive case.*
2. Trace `power(3, 3)`. — *3 × power(3,2) = 3 × 3 × power(3,1) = 3 × 3 × 3 × power(3,0) = 27.*
3. Why does `swap(x, y)` with plain `int` parameters not swap x and y? — *C passes copies; the function swaps the copies, not the originals.*

Part 9 — Arrays and strings (Topic 8.1)

An array is a row of same-type boxes under one name, numbered from **0**; a string is a `char` array that ends with the special character `'\0'`.

Like you're 5. An array is an egg tray. Every hole holds the same kind of thing, and the holes are numbered. The odd part: the first hole is number **0**, not 1. A string is beads on a thread, with a stopper bead at the end so you know where the word finishes. That stopper is `'\0'`.

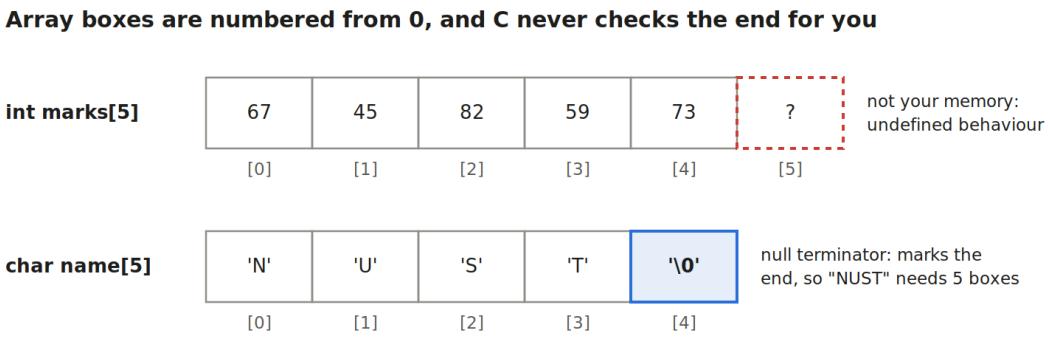


Figure 6 — An int array and a string in memory

An array of 5 has indexes 0 to 4. Index 5 belongs to something else, and C will not stop you from touching it. The C standard leaves out-of-bounds access **undefined**, and attackers have built real exploits on exactly this (paper: *Weird Machines as Insecure Compilation*, arXiv:1911.00157).

Declare, fill, use:

```c
int marks[5] = {67, 45, 82, 59, 73};   /* declare and initialise */
int i, sum = 0, max = marks[0];

for (i = 0; i < 5; i++) {               /* i < 5, never i <= 5 */
    sum += marks[i];
    if (marks[i] > max)
        max = marks[i];
}
printf("Average = %.1f, Highest = %d\n", sum / 5.0, max);  /* 65.2, 82 */
```

Useful facts: `int a[5] = {0};` sets every box to 0. `sizeof(marks) / sizeof(marks[0])` gives the number of elements (5). The array name `marks` on its own is the address of box 0 (Part 10).

2-D arrays — tables and matrices. Engineers use these for stiffness matrices and survey grids:

```c
double K[2][3] = { {4.0, -2.0, 0.0},
                   {-2.0, 4.0, -2.0} };   /* 2 rows, 3 columns */
for (int r = 0; r < 2; r++) {
    for (int c = 0; c < 3; c++)
        printf("%6.1f", K[r][c]);
    printf("\n");
}
```

Strings. `char name[5] = "NUST";` stores N, U, S, T and then `'\0'` automatically, so always allow one extra box. Include `<string.h>` for these helpers:

Function	Does	Example	Result
`strlen(s)`	Length, not counting `'\0'`	`strlen("NUST")`	`4`
`strcpy(d, s)`	Copy s into d	`strcpy(city, "Bulawayo");`	city holds “Bulawayo”
`strcat(d, s)`	Add s onto the end of d	`strcat(city, " CBD");`	“Bulawayo CBD”
`strcmp(a, b)`	Compare; 0 means equal	`strcmp("abc", "abc")`	`0`

The destination array must be big enough, or `strcpy`/`strcat` write past its end. Reading text: `scanf("%s", name);` stops at the first space and has no size limit. Prefer `fgets(name, sizeof name, stdin);`, which limits the length and reads spaces. Never use `gets`; it was removed from the language because it cannot be made safe.

Write `strlen` yourself — a common test question:

```c
int myStrlen(const char s[])
{
    int n = 0;
    while (s[n] != '\0')   /* walk until the stopper bead */
        n++;
    return n;
}
```

Passing an array to a function: the function receives the address, not a copy, so it can change the original, and it does not know the length. Always pass the size too: `double average(int a[], int n)`.

Check yourself

1. `int v[10];` What are the first and last valid indexes? — *0 and 9.*
2. How many `char`s does `"Harare"` need? — *7: six letters plus `'\0'`.*
3. Why must you pass `n` to `average(int a[], int n)`? — *Inside the function the array is only an address, so its length is unknown.*

Part 10 — Pointers and memory

A pointer is a variable that stores a memory **address**: `&` gets an address, `*` follows one. Pointers let functions change your variables, let you walk through arrays, and let you ask for memory while the program runs.

Like you're 5. A pointer is a piece of paper with a house address written on it. The paper is not the house; it tells you where the house is. `&` means "What is your address?" `*` means "Go to that address and look inside." If you change what is inside the house through the paper, the real house changes.

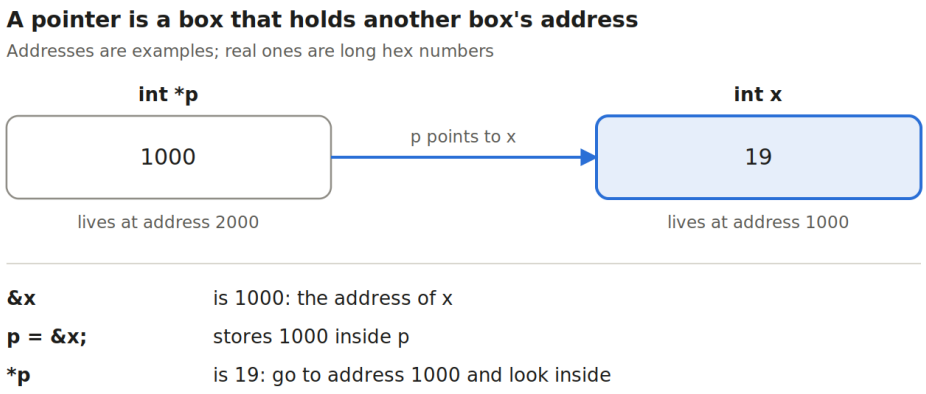


Figure 7 — Pointer p holding the address of x

```c
int x = 19;
int *p = &x;       /* p holds the address of x */
printf("%d\n", *p);  /* 19 */
*p = 25;           /* change x THROUGH the pointer */
printf("%d\n", x);   /* 25 */
int *q = NULL;     /* a pointer to nothing yet; never use *q */
```

Pass by reference — fixing `swap` from Part 8:

```c
void swap(int *a, int *b) { int t = *a; *a = *b; *b = t; }

int m = 3, n = 8;
swap(&m, &n);      /* now m = 8, n = 3 */
```

This is also why `scanf("%d", &n)` needs `&`: `scanf` must write into your variable, so it needs its address.

Pointers and arrays are close cousins. An array name is the address of element 0, and `a[i]` means exactly `*(a + i)`. Adding 1 to a pointer moves it by one *element*, not one byte (4 bytes for an `int`).

```c
int a[3] = {10, 20, 30};
int *p = a;            /* same as &a[0] */
printf("%d %d\n", *(p + 1), p[2]);   /* 20 30 */
```

Dynamic memory — asking for boxes while the program runs. When you only learn the size at run time, use `malloc` from `<stdlib.h>`, and give the memory back with `free`. MIT's 6.087 gives a whole lecture to this (Lecture 11: malloc, valgrind and garbage collection).

```c
#include <stdio.h>
#include <stdlib.h>

int main(void)
{
    int n, i;
    printf("How many readings? ");
    scanf("%d", &n);

    double *r = malloc(n * sizeof(double));   /* n boxes of double */
    if (r == NULL) {                          /* malloc can fail */
        printf("Out of memory\n");
        return 1;
    }
    for (i = 0; i < n; i++)
        scanf("%lf", &r[i]);                  /* use it like an array */

    free(r);                                  /* give it back */
    r = NULL;                                 /* so it can't be reused by mistake */
    return 0;
}
```

The four pointer disasters:

Mistake	What happens	Prevention
Dereferencing `NULL` or an uninitialised pointer	Crash (segmentation fault) or random damage	Initialise every pointer; check for `NULL`
Going past the end of a block	Overwrites other data (a **spatial** error)	Keep the size; loop with `i < n`
Using memory after `free`	Reads or writes memory that now belongs to something else (a **temporal** error)	Set the pointer to `NULL` after `free`
Forgetting `free`	Memory leak: the program keeps growing	Every `malloc` gets exactly one `free`

Researchers split memory safety into exactly these two kinds. *Spatial* safety means never touching memory outside an object's bounds; *temporal* safety means never using memory after it has been released. Microsoft's Checked C project adds bounds-checked pointers to C to stop spatial errors (C to Checked C by 3C, arXiv:2203.13445), and later work extends it to catch use-after-free (Fat Pointers for Temporal Memory Safety of C, arXiv:2208.12900). You don't need these tools for SCS 1101, but they show why examiners keep asking about bounds and `free`.

Check yourself

1. `int y = 7; int *k = &y; *k += 3;` What is y now? — *10.*
2. What does `*(a + 2)` mean for an array a? — *The same as `a[2]`, the third element.*
3. Name the error: `free(p); printf("%d", *p);` — *Use after free, a temporal memory-safety error.*

Part 11 — Structs and simple data structures (Topics 8.1–8.3)

A `struct` bundles related values into one record; data structures are agreed ways of arranging many records so that adding, removing and finding them is fast.

Like you're 5. A `struct` is an ID card: one card, several boxes (name, number, date of birth). Data structures are ways to keep many cards: a **stack** is a pile of plates (you take the top one), a **queue** is the line at the tuck shop (first to arrive is served first), a **linked list** is a treasure hunt where each clue tells you where the next clue is, a **tree** is a family tree, and a **hash table** is a wall of numbered lockers where a rule tells you which locker your name goes in.

Structs:

```c
struct Beam {
    char   id[10];
    double length;   /* m */
    double load;     /* kN/m */
};

struct Beam b1 = {"B1", 6.0, 12.0};
printf("%s: M = %.1f kNm\n", b1.id, b1.load * b1.length * b1.length / 8);  /* dot
operator */

struct Beam *pb = &b1;
pb->length = 7.5;        /* arrow operator: same as (*pb).length */

struct Beam frame[20];   /* an array of 20 beams */
typedef struct Beam Beam;  /* now you can write just Beam */
```

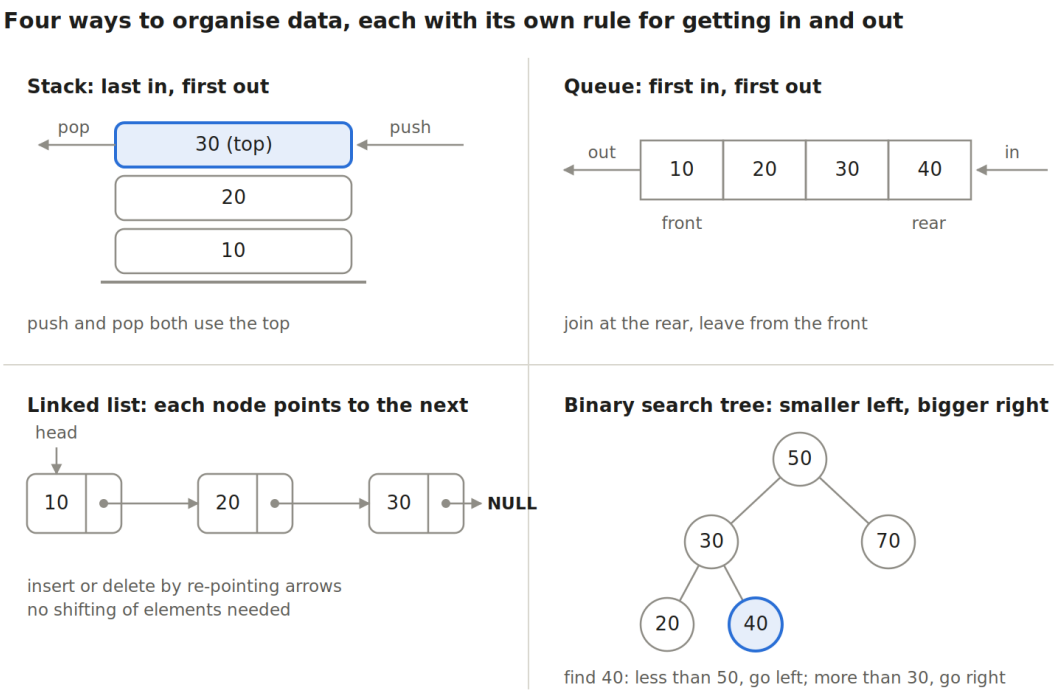


Figure 8 — Stack, queue, linked list and binary search tree

The highlighted plate is the only one you can reach on a stack; the highlighted 40 shows a tree search that checks just 3 nodes instead of all 5.

The structures compared — learn this table:

Structure	Rule	Main operations	Where computers use it
Array / list	Items in order, by position	Read `a[i]` instantly; insert in the middle means shifting	Tables of readings, matrices
Stack	LIFO: last in, first out	`push`, `pop`, `peek` at the top	Undo button, function calls (Part 8), checking brackets
Queue	FIFO: first in, first out	`enqueue` at rear, `dequeue` at front	Printer spooling, FCFS CPU scheduling, keyboard buffer
Linked list	Each node holds data plus a pointer to the next	Insert/delete by changing pointers; must walk from `head` to find	Playlists, memory free-lists, lists that grow and shrink
Binary search tree	Left child smaller, right child bigger	Search, insert in about $\log_2 n$ steps when balanced	File folders, databases, fast lookup
Hash table	A hash function turns a key into a slot number	Insert and find in about 1 step on average	Dictionaries, student-ID lookup, password checks

The queue row links to your SCS 1103 paper: **spooling** puts print jobs in a queue on disk, and **FCFS** scheduling is a queue of processes.

A stack in an array:

```c
#define MAX 100
int stack[MAX], top = -1;          /* -1 means empty */

void push(int x) { if (top < MAX - 1) stack[++top] = x; else printf("Overflow\n"); }
int  pop(void)   { if (top >= 0) return stack[top--]; printf("Underflow\n"); return -
1; }
```

A linked-list node and walking the list:

```c
struct Node { int data; struct Node *next; };

struct Node *newNode(int v) {
    struct Node *n = malloc(sizeof(struct Node));
    n->data = v;  n->next = NULL;
    return n;
}

void printList(struct Node *head) {
    for (struct Node *p = head; p != NULL; p = p->next)
        printf("%d -> ", p->data);
    printf("NULL\n");               /* 10 -> 20 -> 30 -> NULL */
}
```

Hash table idea. With 10 lockers and hash function `h(key) = key % 10`, student ID 20231457 goes to locker 7. If ID 20230997 also lands in locker 7, that is a **collision**; a common fix is to keep a small linked list in each locker (chaining).

Check yourself

1. Push 5, push 8, pop, push 2, pop, pop. What comes out, in order? — *8, 2, 5.*
2. Insert 45, 20, 60, 10, 30 into an empty BST. What are 45's children? — *Left 20, right 60; 10 and 30 hang under 20.*
3. Which structure suits a print queue, and why? — *A queue, because jobs should print in the order they arrived (FIFO).*

Part 12 — Problem solving and algorithms (Topic 9)

An algorithm is a finite, exact, step-by-step method that solves a problem; you plan it with decomposition, pseudocode and flowcharts, then code it. The exam favourites are three sorts and two searches.

Like you're 5. An algorithm is a recipe that never says "add some salt": every step is exact, and it always finishes. Sorting is putting your crayons in order of length. Searching is finding the red crayon: you can check every crayon one by one (linear search), or, if they are already in rainbow order, open the box in the middle and throw away the half that can't have it (binary search).

Problem-solving method (Topic 9.1):

1. **Understand** the problem: inputs, outputs, units, limits.
2. **Decompose** it into small sub-problems (read data, compute, print).
3. **Design** the algorithm in pseudocode or a flowchart.
4. **Code** it, then **test** with normal, boundary and wrong inputs.

Flowchart symbols — often worth easy marks:

Symbol	Shape	Meaning
Terminator	Rounded rectangle (oval)	Start / End
Input/Output	Parallelogram	Read or print data
Process	Rectangle	A calculation or assignment
Decision	Diamond	A yes/no question with two exits
Flow line	Arrow	Order of steps
Connector	Small circle	Joins parts of a chart across a page

The flowcharts in Parts 6 and 7 use exactly these shapes.

Bubble sort — compare neighbours and swap if out of order; the biggest value "bubbles" to the end each pass.

```c
void bubbleSort(int a[], int n) {
    for (int pass = 0; pass < n - 1; pass++) {
        int swapped = 0;
        for (int j = 0; j < n - 1 - pass; j++)
            if (a[j] > a[j + 1]) {
                int t = a[j]; a[j] = a[j + 1]; a[j + 1] = t;
                swapped = 1;
            }
        if (!swapped) break;        /* already sorted: stop early */
    }
}
```

Trace on `5 1 4 2 8`:

Pass	Comparisons and swaps	Array after pass
1	5>1 swap, 5>4 swap, 5>2 swap, 5<8 keep	1 4 2 5 8
2	1<4 keep, 4>2 swap, 4<5 keep	1 2 4 5 8
3	no swaps, so stop	1 2 4 5 8

Selection sort — find the smallest item in the unsorted part and swap it to the front.

```
void selectionSort(int a[], int n) {
    for (int i = 0; i < n - 1; i++) {
        int min = i;
        for (int j = i + 1; j < n; j++)
            if (a[j] < a[min]) min = j;
        int t = a[i]; a[i] = a[min]; a[min] = t;
    }
}
```

Trace on `64 25 12 22 11`: pass 1 → `11 25 12 22 64`; pass 2 → `11 12 25 22 64`; pass 3 → `11 12 22 25 64`; pass 4 → no change.

Insertion sort — take the next card and slide it left into its place among the sorted cards, like sorting a hand of cards.

```
void insertionSort(int a[], int n) {
    for (int i = 1; i < n; i++) {
        int key = a[i], j = i - 1;
        while (j >= 0 && a[j] > key) {
            a[j + 1] = a[j];       /* shift bigger items right */
            j--;
        }
        a[j + 1] = key;
    }
}
```

Trace on `5 2 4 6 1 3`: key 2 → `2 5 4 6 1 3`; key 4 → `2 4 5 6 1 3`; key 6 → no change; key 1 → `1 2 4 5 6 3`; key 3 → `1 2 3 4 5 6`.

Linear search checks every item; it works on unsorted data.

```
int linearSearch(int a[], int n, int target) {
    for (int i = 0; i < n; i++)
        if (a[i] == target) return i;
    return -1;                        /* not found */
}
```

Binary search needs **sorted** data and halves the search area each step.

```
int binarySearch(int a[], int n, int target) {
    int low = 0, high = n - 1;
    while (low <= high) {
        int mid = (low + high) / 2;
```

```c
        if (a[mid] == target) return mid;
        else if (a[mid] < target) low = mid + 1;    /* go right */
        else high = mid - 1;                        /* go left */
    }
    return -1;
}
```

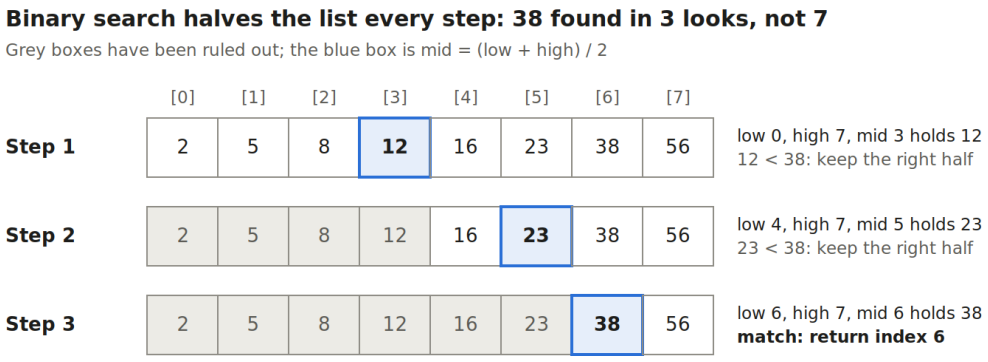


Figure 9 — Binary search for 38: three halving steps on eight sorted values

Each step throws away half of what is left, so 1,000 sorted items need at most about 10 looks, while linear search may need 1,000.

How fast? (Big-O, for n items):

Algorithm	Worst case	Best case	Note
Bubble sort	n^2	n (with the early-stop flag)	Simple, slow on big lists
Selection sort	n^2	n^2	Fewest swaps
Insertion sort	n^2	n	Great for nearly-sorted data
Linear search	n	1	Any data
Binary search	$\log_2 n$	1	Sorted data only

Compression and encryption (Topic 9.4) — the ideas in one example each:

- **Compression** stores the same information in fewer bits. Run-length encoding replaces repeats with a count: `AAAABBBCCD` → `4A3B2C1D`. Lossless formats (ZIP, PNG) give back the exact original; lossy ones (JPEG, MP3) throw away detail people barely notice.
- **Encryption** scrambles data with a key so only key holders can read it. The Caesar cipher shifts each letter: with key 3, `NUST` → `QXVW`. Real systems use strong algorithms such as AES (one shared key) and RSA (a public and a private key).

```c
void caesar(char s[], int k) {
    for (int i = 0; s[i] != '\0'; i++)
        if (s[i] >= 'A' && s[i] <= 'Z')
            s[i] = 'A' + (s[i] - 'A' + k) % 26;
}
```

Numerical algorithms (Topic 9.5). Engineers use them to get answers where no neat formula exists. Newton–Raphson finds √2 by repeating x = (x + 2/x) / 2: starting at 1 it gives 1.5, 1.41667, 1.41422, already correct to 4 decimals. Other examples are the bisection method for roots, the trapezium rule for areas under curves, and Gaussian elimination for simultaneous equations.

Check yourself

1. Why can't you binary-search `9 3 7 1`? — *It isn't sorted, so discarding a half might discard the target.*
2. Show the array after the first pass of bubble sort on `4 3 2 1`. — *`3 2 1 4`*.
3. Encrypt `CIVIL` with a Caesar shift of 2. — *`EKXKN`*.

Part 13 — Files: saving data that outlives the program (Topic 4.4)

Variables vanish when a program ends; files on disk keep data. In C you open a file with `fopen`, read or write it, and always `fclose` it.

Like you're 5. RAM is a whiteboard: fast, but wiped when the lights go off. A file is a notebook in your bag: it is still there tomorrow. Opening a file is opening the notebook to the right page; closing it is putting it safely back in the bag.

The four steps:

```
FILE *fp;                                /* 1. a pointer to a FILE */
fp = fopen("levels.txt", "w");           /* 2. open (here: write) */
if (fp == NULL) { printf("Cannot open file\n"); return 1; }
fprintf(fp, "%d %.2f\n", 1, 2.35);       /* 3. write, just like printf */
fclose(fp);                              /* 4. close: flushes data to disk */
```

Open modes:

Mode	Meaning	If the file doesn't exist	If it exists
`"r"`	Read	`fopen` returns `NULL`	Read from the start
`"w"`	Write	Created	**Wiped**, then written
`"a"`	Append	Created	New data added at the end
`"r+"`	Read and write	Returns `NULL`	Kept
`"rb"`, `"wb"`	Binary versions	as above	as above

Worked program — read borehole water levels from a file and print the average:

```
#include <stdio.h>

int main(void)
{
    FILE *fp = fopen("levels.txt", "r");   /* each line: day level */
    int day, n = 0;
    double level, total = 0.0;

    if (fp == NULL) {
        printf("levels.txt not found\n");
        return 1;
    }
    while (fscanf(fp, "%d %lf", &day, &level) == 2) {  /* stop at end of file */
        total += level;
        n++;
    }
    fclose(fp);
```

```c
    if (n > 0)
        printf("%d readings, average level %.2f m\n", n, total / n);
    return 0;
}
```

Useful file functions: `fprintf` / `fscanf` (formatted text), `fgets` / `fputs` (whole lines), `fgetc` / `fputc` (one character), `fread` / `fwrite` (binary blocks, such as an array of structs), `feof` (end-of-file reached?), `fseek` / `ftell` / `rewind` (move or report the position).

File organisation and access methods (Topic 4.4) — a classic "compare" question:

Method	How records are found	C tools	Good for	Weakness
Sequential	Read from the start, one record after another	`fscanf`, `fgets` in a loop	Payroll runs, logs, processing every record	Slow to find one record in a big file
Random (direct)	Jump straight to a record by its position: record k starts at k × record size	`fseek(fp, k * sizeof(rec), SEEK_SET)` then `fread`	Fixed-size records, booking systems	Needs fixed-size records and a known position
Indexed (indexed-sequential)	Look up the key in a small index, which gives the position, then jump there	An index array of {key, offset} + `fseek`	Student records by ID, library catalogues	Index takes space and must be kept up to date

```c
struct Rec { int id; double mark; } r;
FILE *fp = fopen("marks.dat", "rb");
fseek(fp, 4 * sizeof(struct Rec), SEEK_SET);   /* jump to record 4 (the 5th) */
fread(&r, sizeof(struct Rec), 1, fp);          /* read just that one */
```

Check yourself

1. What does `"w"` do to an existing file? — *It wipes it before writing; use `"a"` to add to it.*
2. Why check `fopen`'s return value? — *It returns `NULL` if the file can't be opened, and using a `NULL` FILE pointer crashes the program.*
3. Differentiate sequential and random access. — *Sequential reads records in order from the start; random jumps straight to a record using its position.*

Part 14 — Bugs, debugging and undefined behaviour: what the research says

Most of your errors will be simple and repeated, but C also has **undefined behaviour**: code the standard gives no meaning to, which may seem to work today and fail tomorrow. Turning on compiler warnings and checkers catches most of both.

Like you're 5. Some mistakes are like spelling errors: the teacher (compiler) circles them in red before you can hand in. Others are like a recipe that says "bake for 2 hours" instead of 20 minutes: the recipe is spelled perfectly, but the cake burns. Undefined behaviour is a recipe step that says "add the thing in the cupboard" when the cupboard is empty: anything could happen, and it might even look fine the first time.

Four kinds of error — a guaranteed exam question:

Kind	When it shows up	Example	Who catches it
Syntax (compile-time)	While compiling	Missing `;`, `prinft`	Compiler error message
Linker	While linking	`undefined reference to 'sqrt'` (fix: add `-lm`)	Linker
Run-time	While running	Division by zero, reading through a `NULL` pointer	Crash, or a checker tool
Logic	Never, automatically	Using `<` where `<=` was needed; wrong formula	You, with test data and trace tables

Undefined behaviour (UB) — learn these six:

1. Reading or writing past the end of an array (Part 9).
2. Using an uninitialised variable (Part 3).
3. Dereferencing a `NULL`, freed or never-set pointer (Part 10).
4. Signed integer overflow, for example `INT_MAX + 1`.
5. Integer division or `%` by zero.
6. Changing a variable twice in one expression, for example `i = i++ + 1;`.

Why it matters (the research). A position paper from MIT CSAIL showed that UB mistakes are subtle, that they occur in real-world systems, sometimes as security holes, and that the usual ways of dealing with them were not enough (Wang, Chen, Cheung, Jia, Zeldovich and Kaashoek, *Undefined Behavior: What Happened to My Code?*, APSys 2012). The same MIT group then built a checker, STACK, which found UB bugs in large systems including the Linux kernel and the Postgres database, because optimising compilers may silently delete code that relies on UB (Wang et al., *A Differential Approach to Undefined Behavior Detection*). On the student side, Fu et al.'s analysis of 53,505 novice compile errors found that a small set of simple error types dominates (ICCE 2015), so a checklist really does catch most of yours.

Your debugging toolkit:

```
gcc -Wall -Wextra -g prog.c -o prog              # warnings on, debug info in
gcc -fsanitize=address,undefined -g prog.c -o prog   # crash with a clear report on
bad memory or UB
valgrind ./prog                                  # finds leaks and bad reads (Linux;
MIT 6.087 Lecture 11)
gdb ./prog                                       # step line by line, print variables
```

No tools? Use **printf debugging**: print the values of key variables inside the loop, and compare them against a hand trace table.

Pre-submission checklist:

- ☐ Every variable initialised before use
- ☐ Every loop has an update that makes its test false eventually
- ☐ Array loops use `i < n`, not `i <= n`
- ☐ Every `scanf` has `&` (except for strings) and the right code (`%lf` for `double`)
- ☐ No division by a value that could be 0
- ☐ Every `fopen` and `malloc` result checked for `NULL`; every `malloc` has a `free`
- ☐ `==` in conditions, not `=`
- ☐ Compiles with `-Wall` and zero warnings

Check yourself

1. Classify: the program prints an average of 72 when it should be 68. — *Logic error.*
2. Why can a UB program "work" on your laptop but fail in the lab? — *The standard promises nothing, so a different compiler, setting or memory layout can give a different result.*
3. Give two causes of a segmentation fault. — *Dereferencing `NULL` and writing far past the end of an array.*

Practice paper with full solutions

Five questions of 25 marks in the same style as the NUST papers you uploaded ("answer any four", define / differentiate / discuss / calculate). Try each question first with a timer: about 45 minutes per question, as in the real 3-hour exam.

Question 1 — Concepts [25]

a) Define, as used in programming: (i) compiler (ii) variable (iii) algorithm (iv) array (v) pointer. [2 each = 10]

b) Using examples, differentiate between a `while` loop and a `do-while` loop. [5]

c) Explain five characteristics of a good algorithm. [10]

Solution 1a (one mark for the idea, one for precision or an example):

- **Compiler:** a program that translates the whole of a high-level source program into machine code before it runs, reporting syntax errors, e.g. GCC for C.
- **Variable:** a named memory location whose value can change while the program runs; it has a type, e.g. `int count = 0;`.
- **Algorithm:** a finite, ordered set of exact steps that solves a problem and terminates, e.g. the steps of binary search.
- **Array:** a collection of elements of the same type stored in consecutive memory under one name and accessed by an index starting at 0, e.g. `double r[10];`.
- **Pointer:** a variable that stores the memory address of another variable, e.g. `int *p = &x;`.

Solution 1b: a `while` loop tests its condition *before* the body, so the body may run zero times; a `do-while` tests *after* the body, so it always runs at least once [2]. Example: `while (n > 0) { ... }` does nothing if n starts at 0 [1.5]; `do { scanf("%d", &c); } while (c < 1 || c > 3);` always asks for input at least once, which suits menus and validation [1.5].

Solution 1c (2 marks each — name + explanation):

1. **Finiteness** — it must stop after a limited number of steps.
2. **Definiteness** — each step is exact and unambiguous ("add 2", not "add a bit").
3. **Input** — it takes zero or more clearly defined inputs.
4. **Output** — it produces at least one result.
5. **Effectiveness** — each step is basic enough to be carried out, by hand or by machine.

Also acceptable: generality (works for all valid inputs) and efficiency (uses little time and memory).

Question 2 — Arrays, sorting and functions [25]

a) Write a C program that reads n marks (n ≤ 100) into an array and prints the average, the highest mark and the number of passes (mark ≥ 50). [12]

b) Show the array after every pass of bubble sort on `7 3 9 1`. [6]

c) With an example, explain pass by value and pass by reference. [7]

Solution 2a:

```c
#include <stdio.h>
#define MAX 100

int main(void)
{
    int n, i, passes = 0;
    double marks[MAX], sum = 0.0, high;

    printf("How many students (1-100)? ");
    scanf("%d", &n);
    if (n < 1 || n > MAX) {                 /* protect the array */
        printf("Invalid number\n");
        return 1;
    }
    for (i = 0; i < n; i++) {               /* input */
        printf("Mark %d: ", i + 1);
        scanf("%lf", &marks[i]);
    }
    high = marks[0];
    for (i = 0; i < n; i++) {               /* process */
        sum += marks[i];
        if (marks[i] > high) high = marks[i];
        if (marks[i] >= 50) passes++;
    }
    printf("Average = %.2f\n", sum / n);    /* output */
    printf("Highest = %.1f\n", high);
    printf("Passes  = %d\n", passes);
    return 0;
}
```

Marking guide: headers and `main` [1], declarations with sensible types [2], input loop with `&` [2], bounds check on n [1], sum/average [2], highest [2], pass count [1], correct output formats [1].

Solution 2b:

Pass	Steps	Array after pass
1	7>3 swap; 7<9 keep; 9>1 swap	3 7 1 9
2	3<7 keep; 7>1 swap	3 1 7 9
3	3>1 swap	1 3 7 9

Solution 2c: In **pass by value** the function receives a copy, so changes inside it do not affect the caller [2]. In **pass by reference** (done in C by passing an address with a pointer) the function works on the original [2]. Example [3]:

```c
void addTenV(int x)  { x += 10; }     /* copy changed only */
void addTenR(int *x) { *x += 10; }    /* original changed */

int a = 5;
addTenV(a);   /* a is still 5 */
addTenR(&a);  /* a is now 15 */
```

Question 3 — Recursion, data structures and decisions [25]

a) Write a recursive function `int sum(int n)` that returns 1 + 2 + … + n, and trace `sum(4)`. [8]

b) Explain a stack and a queue, giving one computing application of each. [7]

c) A water sample is *acidic* if pH < 6.5, *acceptable* if 6.5 ≤ pH ≤ 8.5, and *alkaline* if pH > 8.5. Write a C program that reads a pH and prints the class, rejecting values outside 0 to 14. [10]

Solution 3a:

```c
int sum(int n)
{
    if (n == 0)                 /* base case */
        return 0;
    return n + sum(n - 1);      /* recursive case */
}
```

Trace: sum(4) = 4 + sum(3) = 4 + 3 + sum(2) = 4 + 3 + 2 + sum(1) = 4 + 3 + 2 + 1 + sum(0) = 4 + 3 + 2 + 1 + 0 = **10**. Marks: base case [2], recursive case [2], correct trace showing the calls and the returns [4].

Solution 3b: A **stack** is a LIFO structure: items are added (push) and removed (pop) only at the top, so the last item in is the first out [2]. Application: the call stack that stores function calls and returns, or an editor's undo [1.5]. A **queue** is a FIFO structure: items join at the rear (enqueue) and leave from the front (dequeue) [2]. Application: a printer spool or FCFS process scheduling [1.5].

Solution 3c:

```c
#include <stdio.h>

int main(void)
{
    double ph;
    printf("Enter pH: ");
    scanf("%lf", &ph);

    if (ph < 0 || ph > 14)
        printf("Invalid pH\n");
    else if (ph < 6.5)
        printf("Acidic\n");
    else if (ph <= 8.5)
        printf("Acceptable\n");
    else
        printf("Alkaline\n");
    return 0;
}
```

Marks: input with `%lf` and `&` [2], validation first [2], correct ladder and boundaries [4], output [1], structure and comments [1].

Question 4 — Tracing and debugging [25]

a) What does this print? Show your working. [8]

```
int i, j, count = 0;
for (i = 1; i <= 3; i++)
    for (j = i; j <= 3; j++)
        count += i * j;
printf("%d %d %d\n", i, j, count);

int x = 7;
int y = x++ * 2;
printf("%d %d %d\n", x / 2, x % 3, y);
```

b) Using binary search on `3 8 15 21 30 42 57 66 74`, show the low, high and mid values when searching for (i) 57 and (ii) 10. [7]

c) The program below should read n and print 1 + 2 + … + n. Find and correct five errors. [10]

```
#include <stdio.h>
int main()
{
    int n, i, sum;
    printf("Enter n: ")
    scanf("%d", n);
    for (i = 1; i <= n; i++);
        sum = sum + i;
    if (sum = 0)
        printf("Nothing to add\n");
    printf("Sum is %f\n", sum);
    return 0;
}
```

Solution 4a:

i	j values	Added to count	count after
1	1, 2, 3	1 + 2 + 3	6
2	2, 3	4 + 6	16
3	3	9	25

The outer loop exits when i becomes 4; the last inner loop left j at 4. First line: **`4 4 25`**. Then `y = 7 * 2 = 14` and x becomes 8, so the second line is **`4 2 14`** (8 / 2 = 4, 8 % 3 = 2).

Solution 4b:

Search	low	high	mid	a[mid]	Decision
(i) 57	0	8	4	30	30 < 57, low = 5
	5	8	6	57	found at index 6
(ii) 10	0	8	4	30	30 > 10, high = 3
	0	3	1	8	8 < 10, low = 2

Search	low	high	mid	a[mid]	Decision
	2	3	2	15	15 > 10, high = 1
	2	1	—	—	low > high: not found, return −1

Solution 4c (2 marks per error: find 1, fix 1; six are present):

Line	Error	Correction
`int n, i, sum;`	`sum` not initialised (garbage)	`int n, i, sum = 0;`
`printf("Enter n: ")`	Missing semicolon	`printf("Enter n: ");`
`scanf("%d", n);`	Missing `&`	`scanf("%d", &n);`
`for (...);`	Stray `;` makes an empty loop	Remove the `;` after the `)`
`if (sum = 0)`	Assignment instead of comparison	`if (sum == 0)`
`printf("Sum is %f\n", sum);`	`%f` used for an `int`	`%d`

Question 5 — Structs, files and memory [25]

a) Define a structure for a student (registration number, name, mark) and write a program that reads 3 students and writes them to a text file `students.txt`. [12]

b) Compare sequential, random and indexed file access. [9]

c) Distinguish spatial from temporal memory errors in C, with an example of each. [4]

Solution 5a:

```c
#include <stdio.h>

struct Student {
    char   reg[12];
    char   name[30];
    double mark;
};

int main(void)
{
    struct Student s[3];
    FILE *fp = fopen("students.txt", "w");
    if (fp == NULL) {
        printf("Cannot open file\n");
        return 1;
    }
    for (int i = 0; i < 3; i++) {
        printf("Reg no, surname, mark: ");
        scanf("%11s %29s %lf", s[i].reg, s[i].name, &s[i].mark);  /* widths stop
overflow */
        fprintf(fp, "%s %s %.1f\n", s[i].reg, s[i].name, s[i].mark);
```

```
    }
    fclose(fp);
    printf("Saved 3 records\n");
    return 0;
}
```

Marks: struct definition [3], array of structs [1], `fopen` with `NULL` check [2], input loop (no `&` for strings, `&` for mark) [3], `fprintf` [2], `fclose` [1].

Solution 5b: Use the access-methods table in Part 13: 3 marks per method for how a record is found, one advantage and one weakness.

Solution 5c: A **spatial** error accesses memory outside an object's bounds, e.g. writing `a[10]` in `int a[10]` [2]. A **temporal** error uses memory at the wrong time, e.g. reading through a pointer after `free` [2].

Cheat sheet and 5-week study plan

Match these notes to the outline's week numbers, write and run at least one program every day, and keep this page for the last-night review.

Study plan (weeks follow the SCS 1101 outline; assessment dates are still to be advised):

Week	Outline topics in class	Parts of these notes	Daily practice
1	1–3: computers, number systems, hardware	Parts 1, 3 (binary, ASCII, sizes)	Convert 5 numbers between bases; run `hello.c`
2	4–6: operating systems, networks, databases	Parts 2, 4, 5, 13 (files link to Topic 4.4)	One I/O program a day: read, calculate, print
3	7–8: languages, data structures, recursion	Parts 8, 9, 10, 11	Stack push/pop by hand; trace one recursive call
4	9–10: algorithms, programming practice	Parts 6, 7, 12, 14	Trace a sort by hand, then code it and compare
5	11: presentations, Test 1, revision	Practice paper	Do the practice paper timed: 45 min per question

Templates to know by heart:

```c
/* skeleton */
#include <stdio.h>
int main(void) { /* declare; input; process; output */ return 0; }

/* sum / average / max over an array */
for (i = 0; i < n; i++) { sum += a[i]; if (a[i] > max) max = a[i]; }

/* count matches */
for (i = 0; i < n; i++) if (a[i] >= 50) count++;

/* swap */
t = a; a = b; b = t;

/* validated input */
do { scanf("%d", &x); } while (x < 1 || x > 10);

/* recursion */
if (base case) return answer; return combine(n, f(smaller n));
```

Format codes: `%d` int · `%f` float/double out · `%lf` double in · `%c` char · `%s` string · `%x` hex · `%o` octal · `%.2f` two decimals.

Ten one-liners examiners like:

1. Compiler: whole program before running. Interpreter: line by line while running.
2. Array indexes run from 0 to n − 1.
3. A string needs one extra box for `'\0'`.

4. `7 / 2` is 3; `7 % 2` is 1; `7.0 / 2` is 3.5.
5. `=` assigns; `==` compares.
6. `while` may run 0 times; `do-while` runs at least once.
7. Recursion needs a base case, or the stack overflows.
8. `&x` is the address of x; `*p` is the value p points to.
9. Stack = LIFO; queue = FIFO.
10. Binary search needs sorted data and takes about $\log_2 n$ steps.

How to answer "discuss five" questions for 10 marks: name the point in bold, explain it in one or two sentences, and give an example. That is 2 marks per point, the pattern your SCS 1100 and SCS 1103 papers reward.

Further practice: the MIT 6.087 problem sets with solutions cover data types, operators, linked lists and binary trees at a level just above this course, and your outline also recommends CS50, Khan Academy and Visualgo for animated sorts and searches.

References

Course and university (NUST)

- NUST Faculty of Engineering, *SCS 1101 Introduction to Computer Science and Programming — Course Outline* (lecturer Mr B. Guvava).
- NUST past papers: SCS 1100 *IT and Computer Applications*, December 2024; SCS 1103 *Operating Systems*, November 2024.
- NUST Department of Civil and Water Engineering and the B.Eng. Civil & Water Engineering programme page.
- NUST e-learning (imfundo): https://imfundo.nust.ac.zw · NUST library: http://library.nust.ac.zw (both as listed in the outline).

MIT

- Weller, D. and Chikkerur, S. *6.087 Practical Programming in C*, MIT OpenCourseWare, January IAP 2010 — lecture notes and syllabus. Licence CC BY-NC-SA.
- Wang, X., Chen, H., Cheung, A., Jia, Z., Zeldovich, N. and Kaashoek, M. F. (2012). *Undefined Behavior: What Happened to My Code?* Proceedings of APSys 2012 (MIT CSAIL).
- Wang, X. et al. *A Differential Approach to Undefined Behavior Detection*, ACM Transactions on Computer Systems (the STACK checker).

arXiv papers

- *Weird Machines as Insecure Compilation*, arXiv:1911.00157 — out-of-bounds access as undefined behaviour and the basis of exploits.
- *C to Checked C by 3C*, arXiv:2203.13445 — converting C to bounds-checked Checked C (spatial safety).
- *Fat Pointers for Temporal Memory Safety of C*, arXiv:2208.12900 — extending Checked C to catch use-after-free (temporal safety).

Programming-education research

- Fu, X., Yin, C., Shimada, A. and Ogata, H. (2015). *Error Log Analysis in C Programming Language Courses*. International Conference on Computers in Education (53,505 errors from 909 novice students).

Textbooks recommended in the outline

- Kernighan, B. W. and Ritchie, D. M. (1988). *The C Programming Language*, 2nd ed. Prentice Hall.
- Hanly, J. R. and Koffman, E. B. (2012). *Problem Solving and Program Design in C*, 7th ed. Addison-Wesley.
- Brookshear, J. G. and Brylow, D. (2019). *Computer Science: An Overview*, 13th ed. Pearson.
- Forouzan, B. A. and Fegan, S. C. (2012). *Foundations of Computer Science*, 3rd ed. Cengage.